

AI-Generated Steganographic Malware: Evaluating Detection and Evasion in Modern Anti-Virus Systems

Master of Science in Computer Science at the University of
Johannesburg

TS Siphuma

Supervisor:

Student number: 21700156

Table of Contents

2.1. Introduction	1
2.2. Evolution of Steganography	2
2.2.1. Historical and Traditional Techniques.....	3
2.2.2. Least Significant Bit (LSB) Substitution	5
2.2.3. AI-Generated Steganography	6
2.3. Limitations of Current Antivirus Solutions	7
2.3.1. Ineffectiveness of Signature-Based Detection.....	7
2.3.2. Failure Against Concealed Malware.....	8
2.4. Detection Challenges for Steganographic Malware	9
2.4.1. Ineffectiveness of Existing Tools	9
2.4.2. Evasion Techniques	9
2.4.3. Need for Advanced Algorithms	10
2.5. YARA: Capabilities and Limitations	11
2.5.1. YARA's Role and Capabilities	11
2.5.2. YARA's Limitations	12
2.5.3. The Need for a Hybrid Approach	12
2.6. The Role of VirusTotal in Testing	13
2.7. Emerging Solutions and Approaches	14
2.7.1. Custom YARA Rules	15
2.7.2. Machine Learning-Based Anomaly Detection.....	16
2.7.3. Hybrid Detection Models	16
2.8. Summary	17
References	18

Figures

Figure 1: Classification of spatial domain image steganography techniques

Figure 2: Architecture of the steganography model

Chapter 2: Literature review

2.1. Introduction

In the past years, AI has advanced significantly with the emergence of large generative models like ChatGPT and Stable Diffusion, which can answer questions, generate artistic images, and revolutionize multiple sectors (Gozalo-Brizuela and Garrido-Merchan 2023). Generative AI is now being used by cyber adversaries to lower the skill required for criminal activity, assist in creating malware, and facilitate other malicious actions (Metta, et al. 2024). Some use of AI-generated steganography was by Motwani, et al. (2024) to hide information in the outputs of generative AI models.

Steganography is a Greek word which means concealed writing. The word “steganos” means “covered “ and “graphical “ means “writing” (Kour & Verma, 2014) which is part of the data hiding techniques together with water marking (Rustad et al., 2023) and this paper focus on steganography which Mandal et al. (2022) described Steganography as the art of concealing information in a cover media in such a way that the presence of the information is unknown. Steganography can be classified into following different types such as Text, Images and Audio described by Singh (2016). This study will focus on image steganography, where image files such as JPEG and PNG will be embedded with secret information. It will be explored through techniques such as image format exploitation, spatial domain steganography, frequency domain steganography, and adaptive steganography (Singh, 2016).

The Antivirus software's are primarily and traditionally used to detects and classify malware using signatures which relies on a vendor-generated database, heuristics, rules, and also artificial intelligence (Hussain Hussain and Raheel Abbas 2025). Signature-based detection is a malware detection approach in which at least one byte of the code will be compared to an existing signature of already existing malwares, which are stored in a database known (Chakravarty, et al. 2019).

The chapter synthesizes peer-reviewed papers, industry whitepapers and public datasets from 2000–2025, grouped by technique, detection approach, and operational evaluation. Furthermore, this chapter reviews the literature on steganography and its intersection with artificial intelligence, with the aim of situating AI-generated steganographic malware within current antivirus (AV) detection capabilities (Williams 2019).

2.2. Evolution of Steganography

Steganography, the science that involves communicating secret data in an appropriate multimedia carrier such as an image (Cheddad, et al. 2010). The practice has a rich history and literature which has rapidly developed over the past decades (Sahu and Sahu 2020). Although its roots goes back to ancient times when Demaratus (king of Sparta) and Histiaeus (ruler of Miletus) skilfully communicated using hidden messages (Sahu and Sahu 2020), the recent rise in technological advancement have moved the practice from inks and hidden messages to digital pixels manipulations (Evsutin, et al. 2020) and more recently AI-generated steganography.

Understanding this evolution provides essential context for appreciating current threats, particularly those enhanced by artificial intelligence. Steganography has evolved over time using three key perspectives:

- Carrier modality: Steganography has evolved from ancient methods, such as hiding messages under wax tablets as described by (Sahu and Sahu 2020), to modern techniques that use digital media like images, audio, video, and 3D shapes. Today, advanced frameworks allow multiple data to be hidden across these media types without the need for training deep neural networks, reducing computational cost and avoiding domain adaptation issues (Song, et al. 2024).
- Capacity-detectability trade-offs: A fundamental challenge in steganography has always been the trade-off between hiding capacity and imperceptibility (Sahu and Sahu 2020). Increasing the amount of data hidden within a carrier file generally introduces more statistical distortions, making the presence of the hidden message easier to detect (Cheddad, et al. 2010). Early digital methods, such as simple Least Significant Bit (LSB) substitution, could achieve high capacity but were vulnerable to statistical steganalysis attacks that could easily spot these modifications (Sahu and Sahu 2020). In contrast, more advanced techniques operating in the frequency domain (e.g., DCT, DWT) offered better resistance to detection and image processing but typically at the cost of a lower hiding capacity (Cheddad, et al. 2010). Modern AI-based methods, particularly those using Generative Adversarial Networks (GANs), attempt to resolve this conflict by learning to embed data in a way that mimics the natural statistical properties of the cover image, thereby aiming for higher capacity while maintaining low detectability (Liu, et al. 2020).
- Degree of automation in embedding: The embedding process has evolved from being entirely manual to fully automated and intelligent. Early and traditional digital steganography techniques required significant human intervention or relied on simple, fixed algorithms, such as sequentially replacing the LSBs of pixels (Sahu and Sahu 2020). The next step in automation involved adaptive steganography, where algorithms would automatically analyze the cover image to find more complex and less perceptible locations for data hiding, such as in textured or noisy regions (Cheddad, et al. 2010). The most recent evolution is driven by deep

learning, which has introduced a high degree of automation (Sahu and Sahu 2020). AI models like GANs and Convolutional Neural Networks (CNNs) can automate the entire process by training a neural network to learn the optimal embedding strategy on its own, without needing pre-defined rules for hiding data (Saputro, et al. 2024). For instance, a GAN-based system trains a generator network to produce stego-images that are complex enough to fool a corresponding discriminator network, effectively automating the creation of highly secure and imperceptible steganography (Liu, et al. 2020).

2.2.1. Historical and Traditional Techniques

In today's world, steganography could play a crucial role in ensuring secure and private communications, particularly in environments subject to surveillance and monitoring (Choudhary and Agarwal 2024). This role has actually advanced from ancient times as Sahu and Sahu (2020) explained Emeritus wrote the warning message to Greece about the forthcoming attack inside the piece of wood, which was placed beneath a wax tablet. Likewise, Histiaeus tattooed a message on his messenger's shaved scalp and then waited for the hair to regrow (Diepenbroek) and In World War II, invisible ink and microdots were popular methods of hiding information within letters and photographs (Hammad, et al. 2024).

Digital steganography emerged with the ability to manipulate carrier bits commonly images and audio to embed secret payloads without obvious visible change (Kumar, et al. 2024). Digital image data is a prevalent medium for steganography due to its high redundancy, providing ample capacity for embedding images (Abdalwahid, et al. 2024). This study focusses on Image steganography, which can be categorized into two domains:

- Transform domain which hides information by modifying the frequency components of a media file, such as using discrete cosine or wavelet transforms, making it more robust against compression and attacks than spatial domain methods (Alyousuf, et al. 2020) and
- spatial domain steganography (Mandal, et al. 2022), In spatial domain techniques the secret message is hidden inside the image by applying some manipulation over the different pixels of the image steganography (Swain and Lenka 2014) as shown in the figure below:

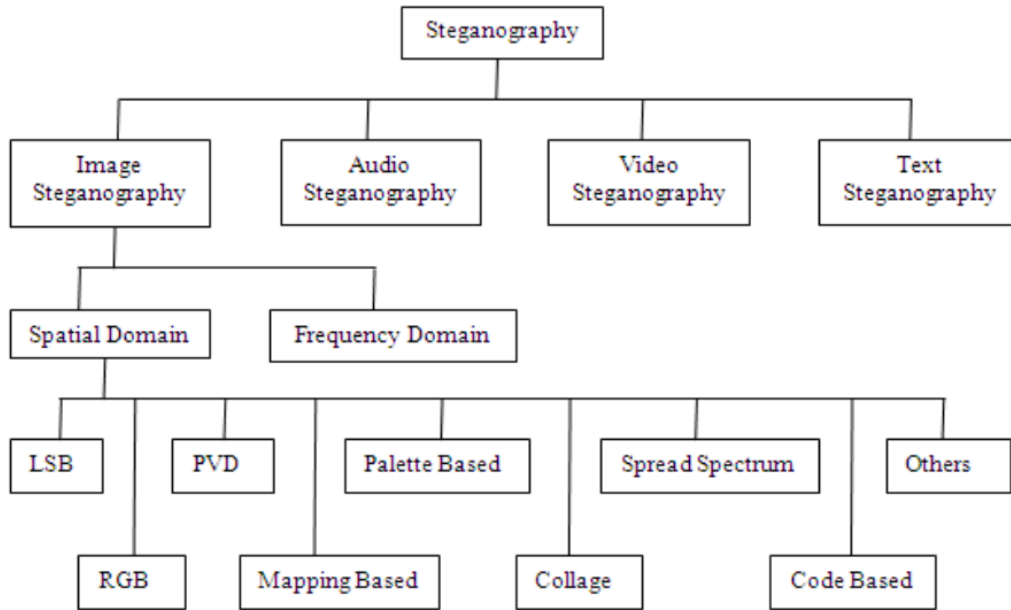


Figure 1: (Swain and Lenka 2014) Classification of spatial domain image steganography techniques

Swain and Lenka (2014) further described spatial domain steganography as encompassing various methods, including LSB which this study will focus on and further be described on

- Least Significant Bit (LSB) : Described by Swain and Lenka (2014), LSB is the simplest and popular image steganographic method is the least significant bit (LSB) substitution and It embeds messages into cover image by replacing the least significant bits directly. To be further under [2.2.2. Least Significant Bit](#).
- pixel value differencing: Mandal, et al. (2022) describes the Pixel Value Differencing (PVD) technique hides secret data by exploiting the fact that edge areas in an image can tolerate more changes than smooth areas and in this method, the cover image is divided into non-overlapping two-pixel blocks, and the secret message is embedded in the difference between the pixel values, calculated as $B_i = k_{i+1} - k_i$ where B ranges from 0 to 255 This study won make use of the PVD as it hides less data, whereas LSB can embed information uniformly across all pixels.
- Other include, RGB-Based which embeds data in the red, green, and blue channels of each pixel using modified LSB techniques to enhance security and minimize visible changes (Swain and Lenka 2014), mapping-based and others.

Alongside embedding methods, there is steganalysis process of detecting, analysing, and potentially extracting hidden information from media, aiming to uncover the presence of steganography (Eid, et al. 2022). Steganalysis is framed by warden models that characterize adversarial which is designed to proactively thwart

steganography by subtly modifying potential cover objects, like images, before they can be used (Al-Obaidi, et al. 2024). This manipulation creates a hostile environment for data hiding, effectively "warding off" secret embedding by causing steganographic algorithms to either fail or produce easily detectable artifacts (Desoky 2011) and passive observers that allows for the monitoring of communication channels to detect hidden information but does not interfere with or alter the data being transmitted (Luo, et al. 2024). This study will follow this passive warden as it analyses files without changing them and the LSB, tools can examine the bit patterns of the least significant bits across the file (Aslam, et al. 2022).

2.2.2. Least Significant Bit (LSB) Substitution

The simplest and popular image steganographic method is the least significant bit (LSB) substitution. It embeds messages into cover image by replacing the least significant bits directly (Swain and Lenka 2014). The LSB is the most well-known algorithm to hide a secret text in an image (Emad, et al. 2018). This study will focus on the LSB on images but this technique can also be employed in other media types to hide data such as audio with such, Audio and video files are considered to be excellent carriers for the purpose of steganography due to presence of redundancy (Asad, et al. 2011).

To reduce detectability, variants such as randomized LSB and LSB matching vary embedding positions and modify insertion logic based on content as ALabaichi, et al. (2020) explained, they further discuss how randomization is a common strategy where hidden information is randomly distributed before being embedded in a cover image to provide better security than sequential methods and this can be achieved by using a "Magic Matrix" to shuffle blocks of an image's blue channel before embedding to enhance security (Rahman, et al. 2025) or chaotic maps to generate secret keys that randomly select pixel locations for data insertion (ALabaichi, et al. 2020). Other methods employ adaptive selection, which targets specific regions of an image, like edges detected via the Canny algorithm, to hide information (Aslam, et al. 2022). Adaptive methods can also vary the number of bits hidden in each pixel to improve the stego-image's quality (Emad, et al. 2018). LSB matching (LSBM) is another widely known variant that alters the embedding logic to be less statistically detectable (ALabaichi, et al. 2020).

LSB methods are vulnerable to statistical detectors histogram analyses and RS tests reveal the subtle distributional shifts LSB introduces shown by Rahman, et al. (2022) histogram analysis is a common detection method used to uncover detailed differences in pixel frequencies between the cover and stego pictures. A high degree of similarity between the histograms indicates minimal distortion, while significant differences can reveal tampering (ALabaichi, et al. 2020). A more powerful and targeted technique is RS steganalysis, a statistical method specifically used to identify the presence of hidden data in LSB-based steganography (Rahman, et al. 2022) . This analysis functions by examining the relationship between "regular" and "singular" pixel groups to detect the subtle changes caused by embedding (Rahman, et al. 2022) . It

is known that standard LSB substitution methods are susceptible to RS analysis (Rahman, et al. 2022) , and many of these simpler techniques are considered easy to break (ALabaichi, et al. 2020).

2.2.3. AI-Generated Steganography

AI-generated steganography involves using neural encoders, generative models, or optimization loops to conceal payloads that is difficult to detect inside digital media as noted by (Raymond and Rick 2023) they continue to mention how the evolution represents a subtle yet potent cyber threat where malicious code is concealed within seemingly innocuous files, rendering traditional cybersecurity defences often powerless (Ali and Bhatti 2024). This technique, also known as coverless steganography, transforms data directly into a photorealistic image rather than hiding it within a pre-existing cover file (Rehman, et al. 2024). Generative models such as Generative Adversarial Networks (GANs) and diffusion models are central to this process, as they are driven by secret information to directly construct the final stego media (Liu, et al. 2020). The competitive dynamic within a GAN serves as an inherent optimization loop, where a generator network refines a carrier's quality and stealth to fool a discriminator, making the resulting steganographic malware difficult to detect (Saputro, et al. 2024).

Fig. 1

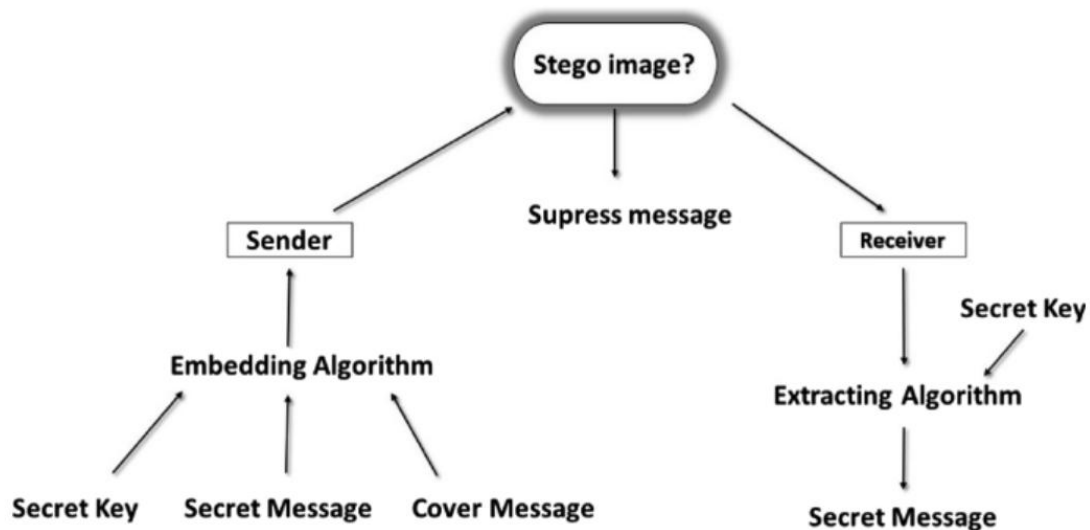


Figure 2 : (Georges and Magdi 2020) Architecture of the steganography model

Encoder-decoder architectures learn mappings between payloads and carrier perturbations, while GANs can synthesize carriers that look natural and embed information within their generation process (Zhang, et al. 2021). Encoder-decoder models can be explicitly trained to embed a secret payload into an image, while a corresponding decoder is trained to extract it, creating a structured method for concealment (Gao, et al. 2022). In contrast, GAN-based concealment produces a stego-image that is visually very natural and difficult to distinguish from an authentic

one (Saputro, et al. 2024). This is a significant departure from traditional techniques like Least Significant Bit (LSB) substitution, which operate at the bit or pixel level by modifying an existing image (Raymond and Rick 2023). AI-powered methods instead embed data at a semantic or structural level, making the concealment an integral part of the media's fabric (Raymond and Rick 2023). This approach avoids leaving the typical modification artifacts that signature-based tools like YARA are designed to find, posing a significant challenge to modern antivirus software (Almomani, et al. 2022).

AI enables embeddings that preserve semantic coherence and can be optimized adversarial against detectors, increasing stealth and cross-detector transferability. A key capability that enhances evasion is operating at a semantic level, which allows malicious data to be embedded within the meaning of the content itself without disrupting its logical structure (Raymond and Rick 2023). One advanced method achieves this by embedding a message in the linguistic domain of an audiovisual file, which preserves semantic equivalence and is inherently robust against spatial or temporal analysis (Chang and Echizen 2025). This semantic plausibility makes it difficult for the machine learning-based anomaly detection proposed in this research to identify statistical irregularities. Furthermore, AI models can be enhanced through adversarial training to improve robustness against various steganalysis tools (Gao, et al. 2022). This results in a higher degree of stealth, with detection rates for some AI-generated images falling to around 50%, which is equivalent to random guessing and illustrates the need for more advanced detection models (Gao, et al. 2022).

Despite their power, AI-driven methods lack standardized benchmarks and raise reproducibility questions, complicating assessments of AV resilience. The primary forensic challenge in detecting AI-generated steganographic malware is that there is often no "original" carrier file for comparison, which undermines traditional detection methods that rely on identifying statistical deviations (Raymond and Rick 2023). This complicates both YARA rule creation and the training of anomaly detection models, which benefit from a clear baseline of what constitutes a "clean" file. This problem is magnified when threat actors use private or fine-tuned AI systems, as it becomes nearly impossible to reverse-engineer the logic to create reliable detection signatures (Raymond and Rick 2023). These gaps directly impact AV systems, which have been shown to fail in detecting malware once it is concealed steganographically (Almomani, et al. 2022). The ability for these techniques to carry viruses and malware presents major risks, justifying the need for a hybrid detection pipeline that combines signature-based and anomaly-based approaches to counter this evolving threat (Williams 2019).

2.3. Limitations of Current Antivirus Solutions

2.3.1. Ineffectiveness of Signature-Based Detection

AI-generated steganography utilizes generative models such as Generative Adversarial Networks (GANs) to craft carriers that conceal payloads while reducing detectable artifacts (Saputro, et al. 2024). This technique is frequently a form of

coverless steganography, where data is transformed directly into a photorealistic image rather than being concealed within a separate cover file (Gao, et al. 2022). GANs leverage a competitive interaction between a generator and a discriminator to produce a stego-image that is very natural and difficult to distinguish from an authentic one (Saputro, et al. 2024). Alternatively, encoder-decoder architectures are explicitly trained to embed secret authentication codes and payloads into an image share, which can then be extracted by a corresponding decoder (Gao, et al. 2022). Unlike traditional methods like LSB that operate at the bit or pixel level, these AI-powered approaches embed data at a semantic or structural level, making the concealment part of the media's fabric (Raymond and Rick 2023). The ability to preserve semantic coherence makes the steganography robust and difficult for steganalysis models to detect (Chang and Echizen 2025). A primary forensic challenge is that there is often no "original" carrier against which the stego media can be compared, which undermines traditional detection techniques (Raymond and Rick 2023). This creates a significant security risk, as AI-generated steganography can carry attacks, viruses, and malware (Williams 2019), with studies showing that antivirus engines often fail to detect malicious payloads once they are concealed within media files (Almomani, et al. 2022).

2.3.2. Failure Against Concealed Malware

Scanning every media file deeply is expensive and latency-sensitive, which leads many AV vendors to apply shallow scanning heuristics to multimedia. This creates significant blind spots for malware concealed within media files such as images and videos. In one study, while many antivirus engines detected a ransomware sample in its original format, none could identify it after it was concealed within an HEVC video stream (Almomani, et al. 2022). Hiding malware in images has similarly been shown to bypass nine out of ten antivirus tools (Dell'Orco, et al. 2025b). This evasion is effective because deep scanning is prone to high rates of false positives, as many legitimate applications perform image manipulation operations that are structurally similar to steganographic decoding (Dell'Orco, et al. 2025b). Attackers exploit this by hiding malicious payloads in inconspicuous-looking carriers like JPEG images to circumvent security software (Knöchel and Karius 2024). Consequently, steganographic tools and the malware they carry can evade perimeter controls like web filters and anti-virus systems (Frith 2007).

A primary advantage of stegomalware is its ability to bypass static analysis because the malicious payload is only revealed at runtime (Dell'Orco, et al. 2025b). This creates a critical time-to-detection problem, as experiments show it can take several days for AV engines to adapt their signatures after a new threat is submitted (Dell'Orco, et al. 2025b). This operational window allows the malware to establish a connection to its Command & Control (C&C) servers to download additional malicious code (Knöchel and Karius 2024). A primary goal for attackers during this period is the stealthy exfiltration of confidential information (Mazurczyk and Caviglione 2014). Malware can be used to steal files and send company secrets out of an organization using

steganographic channels (Frith 2007). Because the malicious activity only commences after the payload is extracted from its benign-looking carrier, behaviour-based detection systems are placed at a disadvantage, reacting to the attack only after the initial infiltration has succeeded.

2.4. Detection Challenges for Steganographic Malware

Anyone may use the steganography techniques to secretly share the data for legitimate purpose, but adversaries constantly look up new attacking ways and may be used steganography for illegitimate attack purpose (Chaganti, et al. 2021).

2.4.1. Ineffectiveness of Existing Tools

There have been increasing use of security software like anti-virus programs in the current systems (Knöchel and Karius 2024). Commercial antivirus solutions primarily rely on signature-based detection, identifying malware only after its signature has been analyzed and catalogued (Dell'Orco, et al. 2025b). As noted, The signature-based approach detects anomalies by comparing system data to the features of known attacks (Kwon, et al. 2022). Since steganography principle is to hide secret information without being noticed by a third party by modifying redundant data in digital media or protocols, such that the carrier's use attributes are not changed during transmission (Wu, et al. 2021).

(Suarez-Tangil, et al. 2014) found in their study that the current malware detection static techniques fail to identify malicious components when they are obfuscated or embedded separately from the code e.g. hidden in an image, they further explain how stegomalware specimen the malicious component is revealed at execution time only. Thus, it will not match any signature even if the search includes the asset where it is hidden which modern antivirus system use. A study by (Badhani and Muttoo 2018) evaluated ten commercial anti-malware programs to see if they could detect malicious applications hidden inside images using various techniques, nine of the ten programs were unable to detect any of the hiding techniques, while the most effective one still failed against malware hidden with encryption or steganography. (Dell'Orco, et al. 2025a) further revealed that antivirus systems are ineffective against stegomalware. Although the proposed solution combines steganalysis with dynamic analysis to successfully detect stegomalware, developing a more general detection mechanism independent of the specific embedding scheme is not trivial.

2.4.2. Evasion Techniques

There are a lot of steganography malware evasion techniques, below are noted by (Ghamizi, et al. 2021) :

- EAST (Evasion Attack Steganography): Regarded as a new steganography and watermarking technique based on multi-label targeted evasion attacks. The key idea of EAST is to encode data as the labels of the image that the evasion attacks produce. Its principle is to encode a message as an adversarial image.
- ADV-EMB: A technique that uses traditional statistical heuristics (UNIWARD) to hide the message then an evasion attack against the detector to update the heuristic to make it less detectable.
- SGAN (Steganographic Generative Adversarial Networks): Introduced SGAN a three-part GAN where the generator is trained simultaneously with a discriminator (to ensure the realism of cover-images) and a steganalyser (i.e a detector that tries to distinguish clean images and stego images) to ensure elusiveness.
- HiDDen: HiDDen proposes to add a differentiable noise layer between the encoder and the decoder and train the whole model at once. This layer ensures that the loss reconstruction considers potential perturbations of the images and generates robust stego images

2.4.3. Need for Advanced Algorithms

The demonstrated ineffectiveness of existing signature-based tools explain on [2.4.1](#) coupled with the development of sophisticated, AI-driven evasion techniques shown on [2.4.2](#) creates a significant security gap. Suarez-Tangil, et al. (2014) showed that traditional antivirus solutions are fundamentally ill-equipped to handle the dynamic and subtle nature of modern stegomalware, as they rely on detecting known patterns that steganography is explicitly designed to conceal which was further explained by Badhani and Muttou (2018).

The evasion methods detailed, such as SGAN and EAST, are not merely hiding data; they are leveraging advanced machine learning concepts like Generative Adversarial Networks and adversarial attacks to actively fool detection systems (Ghamizi, et al. 2021). This represents a paradigm shift where attackers are using intelligent algorithms to create steganographic content that is statistically indistinguishable from legitimate media thus a reactive, signature-based defence is no longer viable.

To counter these threats, there is an urgent need for a new generation of advanced detection algorithms. These algorithms must move beyond simple pattern matching and employ machine learning and deep learning models capable of performing sophisticated steganalysis. Such systems are required to identify the subtle, complex statistical artifacts that even advanced steganographic techniques leave behind. To effectively combat AI-powered attacks, the defence must also be AI-powered, capable of adapting to and identifying novel embedding methods as they emerge.

2.5. YARA: Capabilities and Limitations

YARA is a powerful tool for pattern matching and rule-based classification that has become a critical component in modern malware identification and mitigation strategies (Vinay Patil 2025). It enables security professionals to create custom rules that detect specific patterns in files, memory, or network traffic, moving beyond the limitations of traditional hash-based indicators (NCCIC 2015). Understanding both the extensive capabilities and the inherent limitations of this signature-based approach is essential for using it effectively and for developing a comprehensive, layered defence against evolving cyber threats (Vinay Patil 2025).

2.5.1. YARA's Role and Capabilities

YARA, often described as "The pattern matching Swiss knife for malware researchers," serves as a de-facto standard for malware identification and classification (VirusTotal 2019). Its primary function is to identify and classify malware by creating rules that search for specific patterns, which can be either textual or binary in nature (Lockett 2021). Unlike traditional cryptographic hashing that looks for exact file matches, YARA excels at identifying entire malware families by exploiting common code similarities (NCCIC 2015). Coscia, et al. (2025) explained how YARA rule is composed of a meta section for descriptive information, a strings section where patterns are defined, and a condition section that contains a Boolean expression determining when the rule should trigger a match as further explained by Dittmann, et al. (2024).

Key capabilities of YARA include:

- **Flexibility and Customization:** YARA is widely accepted for its flexible and customizable nature, which allows malware analysts to develop rules tailored to the requirements of a specific security domain (Naik, et al. 2020). Rules can be crafted with great precision using text strings, hexadecimal strings, and regular expressions to define patterns (Coscia, et al. 2025).
- **Platform Support and Integration:** It is an open-source, multi-platform tool that works on Windows, Linux, and Mac OS explained by Naik, et al. (2020) and further noted that YARA can be integrated as a library into other applications and programming languages, including popular antivirus engines like ClamAV, Python, and C/C++.
- **Versatility in Analysis:** The tool can be used for both static and dynamic malware analysis (Naik, et al. 2020). It is effective at detecting obfuscated malware, as rules can be built to identify specific obfuscation techniques like packing or anti-debugging capabilities (Lockett 2021).

- Extensibility: For conditions that are too complex for the default syntax, YARA's functionality can be extended with custom modules that provide additional metrics and functions not available in the default toolset (Dittmann, et al. 2024).

2.5.2. YARA's Limitations

Despite its widespread use and capabilities, YARA is not a complete solution and has several limitations, many of which are inherent to its nature as a signature-based detection tool (Lockett 2021).

- Evasion by Attackers: The effectiveness of YARA rules is dependent on their defined Indicator of Compromise (IoC) strings, but attackers can manipulate, encrypt, or replace these strings to evade detection (Naik, et al. 2020). Static rules that rely on fixed byte sequences are often ineffective against polymorphic or metamorphic malware, which is designed to constantly change its code structure to avoid signature matching (Vinay Patil 2025).
- Rule Creation and Maintenance: Writing high-quality, effective YARA rules is a tedious process that requires a highly specialized skill-set and expert knowledge of malware analysis. As new malware variants are constantly developed, the continuous maintenance, refinement, and updating of rule sets is a complex and time-consuming task (Khalid, et al. 2020).
- Balancing False Positives and Negatives: Crafting a robust rule involves a difficult trade-off between sensitivity and precision. A rule that is too specific might miss new malware variants, while a rule that is too general can produce a high number of false positives by incorrectly flagging benign files (Vinay Patil 2025). The success of YARA depends on achieving a challenging balance of the right types and number of IoC strings (Naik, et al. 2020).
- Detection of Novel Threats: YARA is most effective in detecting malware that is like existing families and variants. It may fail to detect entirely new or unique malware strains for which no signatures have yet been created (Naik, et al. 2020).
- Automated Rule Generation: While tools that automatically generate YARA rules exist, their output often requires manual post-processing and optimization to be effective and may not perform as well as rules created manually by an expert analyst (Naik, et al. 2020).

2.5.3. The Need for a Hybrid Approach

The limitations of YARA as a standalone, signature-based tool highlight the necessity of adopting a hybrid strategy that combines YARA with other detection techniques to create a more resilient and effective defence (Vinay Patil 2025). Since static YARA

rules can be bypassed, especially by evasive malware, they must be augmented with more dynamic and flexible methods (Vinay Patil 2025).

One proposed hybrid solution is to enhance YARA rules by integrating them with fuzzy hashing (Naik, et al. 2020). Fuzzy hashing complements YARA by identifying structural similarities between files, which is effective even when specific strings or byte sequences have been modified (Naik, et al. 2020). This combined search can increase the accuracy of detection and the overall confidence in the analysis results as (Naik, et al. 2020) explained and further research has also suggested building fuzzy hashing functionality directly into YARA rules to strengthen them (Lockett 2021).

Furthermore, the weakness of signature-based detection against new and evasive threats highlights the need to incorporate behaviour-based analysis (Lockett 2021). A hybrid strategy that combines YARA's static pattern matching with dynamic analysis which focuses on runtime actions such as suspicious API calls, system modifications, and network activity is far more effective at spotting complex risks (Vinay Patil 2025). Finally, research is exploring the integration of machine learning with YARA to automatically generate more adaptive and intelligent rules, helping to overcome the difficulties and time constraints of manual rule creation (Vinay Patil 2025).

2.6. The Role of VirusTotal in Testing

VirusTotal is a free online tool that one of the most popular malware scanning platforms (Leka, et al. 2022) which plays a crucial role in the testing and analysis of potentially malicious files and URLs (Masri and Aldwairi 2017). Its primary function is to analyse submissions using a vast collection of different antivirus engines, website scanners, and domain blacklists to detect various types of malwares, including viruses, worms, and trojans (Masri and Aldwairi 2017). Developed by VirusTotal, the it contains the YARA framework as act as a testament to the organization's central role in the malware research community (Dittmann, et al. 2024). In experimental and research settings, VirusTotal is frequently used as a definitive source to validate whether samples are malicious and to verify their correct classifications. For example, one can script the upload of a URL or binary and retrieve the “detection ratio” (number of engines that label it malicious) (Lockett 2021). Its extensive database of known malware hashes and domains makes it invaluable for validation: researchers frequently cross-reference their samples against VirusTotal to ensure correct labelling (Masri and Aldwairi 2017).

Beyond manual uploads, and apart from its primary public interface, PC uploaders, browser extensions, and an Application Programming Interface (API) are all viable avenues for the transmission of data to VirusTotal (Vasani, et al. 2023). A system can be designed to automatically submit URLs to VirusTotal and parse the detection ratio to classify them as malicious or benign (Masri and Aldwairi 2017). Its extensive and reliable threat database contain utilizes more than 70 anti-malware analysers and URL/domain blacklisting services in its inspections, in addition to other methods for discriminating against dubious matters (Vasani, et al. 2023). However, while

indispensable for testing, its performance is not infallible as Leka, et al. (2022) explained there is no clear understanding of how this platform delivers its results and since the platform operates basically as a Blackbox, the produced results may be questionable. In one study evaluating its effectiveness in detecting malicious advertisements by Masri and Aldwairi (2017) VirusTotal achieved a testing accuracy of 56%, which, while significant, was outperformed by other tools such as URLvoid with 73% that incorporated a wider range of domain blacklists in addition to scanning engines and highlight that while VirusTotal is a powerful and widely used testing platform, its effectiveness can be enhanced when used as part of a broader, multi-tool testing strategy.

The selection of VirusTotal as the primary validation tool for this study is justified by it been a free and accessible online service, it presents a simple yet powerful solution for large-scale analysis, eliminating the complexities and costs associated with acquiring and maintaining dozens of individual antivirus products. This study will leverage its Application Programming Interface (API) to systematically upload AI-generated steganographic samples containing hidden payloads. The resulting "detection ratio" metric indicating how many of the 70+ integrated, industry-leading antivirus engines flag a sample as malicious will serve as the core measure of evasion success. A low detection ratio will provide powerful, empirical evidence of the AI steganography's ability to bypass the current threat detection landscape. Therefore, VirusTotal's comprehensive and standardized approach provides a credible, large-scale benchmark that is essential for strengthening the experimental validation component of this research and solidifying its conclusions on vulnerabilities within modern antivirus systems.

2.7. Emerging Solutions and Approaches

Using steganography during an attack is relatively easy but protecting against it is much more complicated as threat actors are getting more innovative and more creative (Kaspersky 2023). The core challenge lies in the fact that steganography's primary goal is to hide the very existence of a secret message, making it difficult to detect and even more challenging to understand but they do leave behind detectable traces in the cover medium (Provos and Honeyman 2003).

Traditionally, antivirus systems use signature detection, which relies on a vendor-generated database (Hussain Hussain and Raheel Abbas 2025). this method is only effective against known threats. With the rapid evolution of malware, particularly zero-day attacks, signature-based detection often falls short in identifying new or mutated threat (Hussain Hussain and Raheel Abbas 2025). This steganography vulnerability has necessitated the development of more sophisticated and multi-layered defence frameworks (Prathyusha, et al. 2025) and some emerging solutions are moving beyond simple signature matching to embrace more dynamic and intelligent methods of analysis, focusing on three key areas such as:

- The targeted pattern-matching of custom YARA rules,
- The statistical anomaly detection of machine learning, and
- The synergistic power of hybrid models.

2.7.1. Custom YARA Rules

YARA is a tool aimed at (but not limited to) helping malware researchers to identify and classify malware samples (Ningappa 2020). It functions by creating custom rules that match specific, predefined patterns within a file's binary or text content (Vinay Patil 2025). This signature-based detection approach is a fundamental technique used by antivirus and malware analysis software (Lockett 2021). The value of custom rules lies in their ability to be tailored to the unique characteristics of specific threats, such as the subtle artifacts left by steganography (Khalid, et al. 2020). While YARA is a widely accepted and flexible tool for identifying known patterns, its effectiveness is limited against polymorphic malware, which is designed to constantly change its signature to evade detection (Vinay Patil 2025). Despite this limitation, custom YARA rules remain an essential and efficient tool for threat hunting and providing a rapid method for identifying known malicious indicators (NCCIC 2015).

The study will follow process described by Naik, et al. (2020) who explained how YARA at its core is designed to spot malware by matching its characteristics, or "signatures," against a database of known threats. They further explain how it works by scanning files and processes for these tell-tale signs and built with three main parts: meta, strings, and condition. Below is an example of a simple YARA Rule syntax to be employed in the study (Naik, et al. 2021):

```
rule RuleName {
  meta: description = "descriptions of rule"
  author = "name"
  date = "dd/mm/yyyy"
  reference = "url"

  strings:
  $text_string1 = "text1 you wish to find in malware"
  $text_string2 = "text2 you wish to find in malware"
  $hex_string1 = {hex1 you wish to find in malware}
  $hex_string2 = {hex2 you wish to find in malware}
  $reg_exp_string1 = /regular expressions1 you wish to find in malware/
  $reg_exp_string2 = /regular expressions2 you wish to find in malware/

  condition:
  $text_string1 or $text_string2 or
  $hex_string1 or $hex_string2 or
  $reg_exp_string1 or $reg_exp_string2
```

}

(Regeciova 2022) explains the different sections involved in detail from the strings section is where patterns been hunted are defined which can further fall into three categories :

1. **Text Strings:** Simple, readable text. You can add modifiers to make the search case-insensitive (nocase) or specify the character type (ascii, wide).
2. **Hexadecimal Strings:** Used for matching specific sequences of raw bytes, which is great for finding non-textual data. This format is flexible, allowing for wildcards and other advanced patterns.
3. **Regular Expressions:** A very powerful option, available since version 2.0, that allows you to define complex and variable patterns.

The final piece they explained is the condition section. This is the rule's brain, using a logical statement e.g. if 2 of the 3 strings are found, then it's a match to decide whether to officially flag the file as malware. It's this condition that ultimately determines if the rule triggers.

2.7.2. Machine Learning-Based Anomaly Detection

To overcome the limitations of static signatures, emerging solutions are increasingly turning to machine learning (ML) (Adhav, et al. 2024). Machine learning gives computers the ability to learn from data without being explicitly programmed, allowing them to discern complex patterns that traditional methods miss and by utilizing unsupervised learning, a system can analyse unlabelled data to discover hidden patterns and classify outliers on its own (Naeem, et al. 2023). ML-based anomaly detection identifies threats by detecting behaviours and observations that deviate significantly from a baseline of normal activity, rather than looking for a known signature (Zehra, et al. 2023). This approach is particularly effective against unknown or "zero-day" threats because it does not rely on prior knowledge of a specific attack. This makes ML-based anomaly detection a powerful method for flagging suspicious files based on statistical irregularities and unusual behaviours that might indicate concealed malware (Stoian 2020).

Machine learning techniques are based on explicit or implicit model that enables the patterns analysed to be categorized into Genetic Algorithms, Fuzzy Logic, Neural Networks, Bayesian networks and outlier detection:

2.7.3. Hybrid Detection Models

Recognizing the individual weaknesses of signature-based and purely behavioural systems, the most robust emerging solution is the development of hybrid detection models (Vinay Patil 2025). A hybrid approach combines the speed and efficiency of

signature-based tools like YARA for known threats with the adaptability of machine learning for unknown and evasive malware (Adhav, et al. 2024). This layered strategy creates a more comprehensive defence, using different techniques to cover a wider range of attack vectors (Vinay Patil 2025) and Such an integrated framework, combining static analysis with dynamic or ML-based methods, is designed to improve overall detection accuracy while simultaneously reducing the rate of false positives. By leveraging the complementary strengths of each method, a hybrid model can provide a more resilient and effective defence against the evolving threat landscape (Naik, et al. 2020).

2.8. Summary

The literature review systematically examines the existing body of knowledge on steganography and malware detection to provide the foundational context for this study. The review traces the evolution of steganography, highlighting its progression from historical practices to modern, AI-driven digital concealment techniques that present significant detection challenges. A central theme identified is the significant limitation of current antivirus solutions, which are often based on conventional signature-based detection and frequently fail to identify malware once it is concealed within multimedia files. This critical detection gap is substantiated by studies demonstrating that even known malware, such as ransomware, becomes invisible to dozens of antivirus engines after being embedded using steganographic techniques.

The review also assesses the capabilities and inherent limitations of YARA, a key tool for pattern-based malware detection. While YARA offers flexibility in developing custom rules, its effectiveness as a signature-based tool is inconsistent against the evasive and evolving nature of steganographic threats.

Finally, the review explores emerging solutions that aim to overcome these challenges. There is a clear trend towards more adaptive systems, with a strong emphasis on machine learning-based anomaly detection as a promising approach for identifying concealed threats through statistical irregularities rather than known signatures. The findings confirm a distinct gap in current detection methodologies and strongly support the rationale for this study's proposed hybrid model, which integrates enhanced YARA rules with machine learning to counter the threat of AI-generated steganographic malware.

References

Abdalwahid, Shadan MJa, et al.

2024 Investigating the Effectiveness of Artificial Intelligence in Watermarking and Steganography for Digital Media Security. 2024 21st International Multi-Conference on Systems, Signals & Devices (SSD), 2024, pp. 552-561. IEEE.

Adhav, Savita, et al.

2024 Tackling the Emerging Threat of Image-Based Steganographic Malware with Advanced Machine Learning Techniques. 2024 5th International Conference on Data Intelligence and Cognitive Informatics (ICDICI), 2024, pp. 74-79. IEEE.

Al-Obaidi, Sumia Abdulhussien Razooqi, Mina Zolfy Lighvan, and Mohammad Asadpour

2024 Enhanced image steganalysis through reinforcement learning and generative adversarial networks. Intelligent Decision Technologies 18(2):1077-1100.

ALabaichi, Ashwak, Maisa'a Abid Ali K Al-Dabbas, and Adnan Salih

2020 Image steganography using least significant bit and secret map techniques. International journal of electrical & computer engineering (2088-8708) 10(1).

Ali, Atif, and Baber Majid Bhatti

2024 Spies in the Bits and Bytes: The Art of
Cyber Threat Intelligence: CRC Press.

Almomani, Iman, Aala AlKhayer, and Walid El-Shafai

2022 Novel Ransomware Hiding Model Using
HEVC Steganography Approach. Computers,
Materials & Continua 70(1).

Alyousuf, Farah Qasim Ahmed, Roshidi Din, and Alaa
Jabbar Qasim

2020 Analysis review on spatial and transform
domain technique in digital steganography. Bulletin
of Electrical Engineering and Informatics 9(2):573-
581.

Asad, Muhammad, Junaid Gilani, and Adnan Khalid

2011 An enhanced least significant bit
modification technique for audio steganography.
International Conference on Computer Networks
and Information Technology, 2011, pp. 143-147.
IEEE.

Aslam, M. A., et al.

2022 Image Steganography using Least
Significant Bit (LSB) - A Systematic Literature
Review. 2022 2nd International Conference on
Computing and Information Technology (ICCIT),
2022, pp. 32-38.

Badhani, Shikha, and Sunil K Muttoo

2018 Evading android anti-malware by hiding malicious application inside images. International Journal of System Assurance Engineering and Management 9(2):482-493.

Chaganti, Rajasekhar, et al.

2021 Stegomalware: A systematic survey of MalwareHiding and detection in images, machine LearningModels and research challenges. arXiv preprint arXiv:2110.02504.

Chakravarty, Adit Kumar, et al.

2019 A study of signature-based and behaviour-based malware detection approaches. Int. J. Adv. Res. Ideas Innov. Technol 5(3):1509-1511.

Chang, Ching-Chun, and Isao Echizen

2025 Steganography Beyond Space-Time With Chain of Multimodal AI Agents. arXiv:2502.18547.

Cheddad, Abbas, et al.

2010 Digital image steganography: Survey and analysis of current methods. Signal processing 90(3):727-752.

Choudhary, Utkarsh, and Parul Agarwal

2024 Image Steganography Combined with Cryptography for Covert Communication. Proceedings of the 2024 Sixteenth International Conference on Contemporary Computing, 2024, pp. 207-212.

Coscia, Antonio, et al.

2025 APIARY: An API-based automatic rule generator for yara to enhance malware detection. Computers & Security 153:104397.

Dell'Orco, Danilo, et al.

2025a Would you mind hiding my malware? Building malicious Android apps with StegoPack. Pervasive and Mobile Computing:102060.

Dell'Orco, Danilo, et al.

2025b Challenging Antivirus against Elusive Android Malware over Time. Proceedings of the 2025 Italian Conference on Cybersecurity, ITASEC. CEURWS. org, 2025b.

Desoky, Abdelrahman

2011 Edustega: an education-centric steganography methodology. International Journal of Security and Networks 6(2-3):153-173.

Diepenbroek, Martine

Secretive Spartans: Herodotus' views towards secret communication in Persia and Sparta and its effect on post-Herodotean sources.

Dittmann, Jana, et al.

2024 Forensic trace analysis for MP3 based stego-malware: Exemplary study for stego-algorithm and capacity attribution to derive YARA rules for malware identification. Proceedings of the

2024 ACM Workshop on Information Hiding and Multimedia Security, 2024, pp. 101-112.

Eid, Wafa M, et al.

2022 Digital image steganalysis: current methodologies and future challenges. Ieee Access 10:92321-92336.

Emad, Elshazly, et al.

2018 A secure image steganography algorithm based on least significant bit and integer wavelet transform. Journal of Systems Engineering and Electronics 29(3):639-649.

Evsutin, Oleg, Anna Melman, and Roman Meshcheryakov

2020 Digital steganography and watermarking for digital images: A review of current research directions. IEEE Access 8:166589-166611.

Frith, David

2007 Steganography approaches, options, and implications. Network Security 2007(8):4-7.

Gao, Kai, et al.

2022 Steganographic secret sharing via AI-generated photorealistic images. EURASIP Journal on Wireless Communications and Networking 2022(1):119.

Georges, Jeanne, and Dalia A Magdi

2020 Using artificial intelligence approaches for image steganography: a review. Internet of Things—Applications and Future: Proceedings of ITAF 2019:239-247.

Ghamizi, Salah, et al.

2021 Evasion attack steganography: Turning vulnerability of machine learning to adversarial attacks into a real-world application. Proceedings of the IEEE/CVF International conference on computer vision, 2021, pp. 31-40.

Gozalo-Brizuela, Roberto, and Eduardo C Garrido-Merchan

2023 ChatGPT is not all you need. A State of the Art Review of large Generative AI models. arXiv preprint arXiv:2301.04655.

Hammad, Omar Mejbil, et al.

2024 An Overview of Image Steganography Techniques: Historical Development, Methodologies, and Evaluation Criteria. SHIFRA 2024:74-87.

Hussain Hussain, and Raheel Abbas

2025 Evaluating antivirus software: a comparative analysis of detection methodologies and performance metrics in modern antivirus solutions.

Kaspersky

2023 What is steganography? Definition and explanation.

Khalid, Myra, et al.

2020 Automatic YARA rule generation. 2020 International Conference on Cyber Warfare and Security (ICCWS), 2020, pp. 1-5. IEEE.

Knöchel, Mandy, and Sebastian Karius

2024 Text Steganography Methods and their Influence in Malware: A Comprehensive Overview and Evaluation. Proceedings of the 2024 ACM Workshop on Information Hiding and Multimedia Security, 2024, pp. 113-124.

Kumar, Arvind, et al.

2024 Steganography: Unveiling techniques and research agenda.

Kwon, Hee-Yong, Taesic Kim, and Mun-Kyu Lee

2022 Advanced intrusion detection combining signature-based and behavior-based detection methods. Electronics 11(6):867.

Leka, Christian, et al.

2022 A comparative analysis of virustotal and desktop antivirus detection capabilities. 2022 13th International Conference on Information, Intelligence, Systems & Applications (IISA), 2022, pp. 1-6. IEEE.

Liu, Jia, et al.

2020 Recent advances of image steganography with generative adversarial networks. IEEE Access 8:60575-60597.

Lockett, Adam

2021 Assessing the effectiveness of yara rules for signature-based malware detection and classification. arXiv preprint arXiv:2111.13910.

Luo, Weiqi, et al.

2024 A Comprehensive Survey of Digital Image Steganography and Steganalysis. APSIPA Transactions on Signal and Information Processing 13(1).

Mandal, Pratap Chandra, et al.

2022 Digital image steganography: A literature survey. Information sciences 609:1451-1488.

Masri, Rima, and Monther Aldwairi

2017 Automated malicious advertisement detection using virustotal, urlvoid, and trendmicro. 2017 8th International Conference on Information and Communication Systems (ICICS), 2017, pp. 336-341. IEEE.

Mazurczyk, Wojciech, and Luca Caviglione

2014 Steganography in modern smartphones and mitigation techniques. IEEE Communications Surveys & Tutorials 17(1):334-357.

Metta, Shivani, et al.

2024 Generative AI in cybersecurity. arXiv preprint arXiv:2405.01674.

Motwani, Sumeet, et al.

2024 Secret collusion among AI agents: Multi-agent deception via steganography. Advances in Neural Information Processing Systems 37:73439-73486.

Naeem, Samreen, et al.

2023 An unsupervised machine learning algorithms: Comprehensive review. International Journal of Computing and Digital Systems.

Naik, Nitin, et al.

2020 Evaluating automatically generated YARA rules and enhancing their effectiveness. 2020 IEEE Symposium Series on Computational Intelligence (SSCI), 2020, pp. 1146-1153. IEEE.

Naik, Nitin, et al.

2021 Embedded YARA rules: strengthening YARA rules utilising fuzzy hashing and fuzzy rules for malware analysis. Complex & Intelligent Systems 7(2):687-702.

NCCIC

2015 USING YARA FOR MALWARE DETECTION.1.

Ningappa, Siddesh

2020 YARA Based Defence Mechanism Against NFC Based Attacks For Android Devices, Dublin, National College of Ireland.

Prathyusha, Kema, J Vinoj, and KB Manikandan

2025 Exploring Artificial Intelligence Security: A Comparative Study of Adversarial Attacks and Steganographic Defenses. 2025 4th International Conference on Sentiment Analysis and Deep Learning (ICSADL), 2025, pp. 20-25. IEEE.

Provos, Niels, and Peter Honeyman

2003 Hide and seek: An introduction to steganography. IEEE security & privacy 1(3):32-44.

Rahman, Shahid, et al.

2025 A novel and efficient digital image steganography technique using least significant bit substitution. Scientific Reports 15(1):107.

Rahman, Shahid, et al.

2022 A novel steganography technique for digital images using the least significant bit substitution method. IEEE Access 10:124053-124075.

Raymond, Jessica, and Lopez Rick

2023 AI-POWERED STEGANOGRAPHY: HIDDEN COMMUNICATIONS IN GENERATIVE MEDIA ON THE DARKNET.

Regeciova, Dominika

2022 Yara: Down the Rabbit Hole Without Slowing Down. The Journal on Cybercrime and Digital Investigations 7(1):17-21.

Rehman, Hafiz Abdul, et al.

2024 Leveraging coverless image steganography to hide secret information by generating anime characters using GAN. Expert Systems with Applications 248:123420.

Sahu, Aditya Kumar, and Monalisa Sahu

2020 Digital image steganography and steganalysis: A journey of the past three decades. Open Computer Science 10(1):296-342.

Saputro, Indrawan Ady, et al.

2024 AI-Powered Steganographic Techniques: A Comparison of Traditional Methods and Modern Machine Learning Approaches. 2024 6th International Conference on Cybernetics and Intelligent System (ICORIS), 2024, pp. 01-06. IEEE.

Song, Sojeong, et al.

2024 Implicit Steganography Beyond the Constraints of Modality. European Conference on Computer Vision, 2024, pp. 289-304. Springer.

Stoian, Nicolas-Alin

2020 Machine learning for anomaly detection in iot networks: Malware analysis on the iot-23 data set, University of Twente.

Suarez-Tangil, Guillermo, Juan E Tapiador, and Pedro Peris-Lopez

2014 Stegomalware: Playing hide and seek with malicious components in smartphone apps. International conference on information security and cryptology, 2014, pp. 496-515. Springer.

Swain, Gandharba, and Saroj Kumar Lenka

2014 Classification of image steganography techniques in spatial domain: a study. Journal of Computer Science & Engineering Technology (IJCSET) 5(03):219-232.

Vasani, Vatsal, et al.

2023 Comprehensive analysis of advanced techniques and vital tools for detecting malware intrusion. Electronics 12(20):4299.

Vinay Patil, Naveen Kumar M, Pawan Singh M, Ashishika Singh

2025 Effectively Writing YARA Rules to Detect Malware.

VirusTotal

2019 YARA in a nutshell.

Williams, Octavia C

2019 What are the cybersecurity risks of artificial intelligence generated steganography?, Utica College.

Wu, Zhijun, et al.

2021 Steganography and steganalysis in voice over IP: A review. Sensors 21(4):1032.

Zehra, Sehar, et al.

2023 Machine learning-based anomaly detection in NFV: A comprehensive survey. Sensors 23(11):5340.

Zhang, Chaoning, et al.

2021 A brief survey on deep learning based data hiding. arXiv preprint arXiv:2103.01607.